

Anexo 9.1 Consultas en Cassandra

En este apartado se presenta el modelo de datos implementado en Cassandra para almacenar una muestra del dataset de clientes y ejecutar consultas orientadas al análisis del churn. La sección se organiza en cuatro bloques: configuración del entorno, creación de estructuras, inserción de datos y análisis de resultados.

El acceso a Cassandra se realizó mediante la plataforma Killercode, utilizando un entorno Ubuntu preparado para ejecutar contenedores Docker:

<https://killercode.com/playgrounds/scenario/ubuntu>

Una vez iniciado el entorno de trabajo, se accedió a la consola de Cassandra para ejecutar los comandos necesarios: creación del keyspace, definición de la tabla, inserción manual de registros y realización de consultas filtradas.

Objetivo de la configuración: preparar un entorno reproducible con Cassandra 3.11 mediante Docker, de forma que las consultas puedan ejecutarse desde la consola CQL.

1. Descarga de la imagen de Cassandra.

```
docker pull cassandra:3.11
```

2. Creación de la red Docker.

```
docker network create cassandra
```

3. Ejecución del contenedor de Cassandra.

```
docker run --rm -d \
```

```
--name cassandra \
```

```
--hostname cassandra \
```

```
--network cassandra \
```

```
cassandra:3.11
```

4. Apertura de la consola CQL.

```
docker run --rm -it \  
--network cassandra \  
nuvo/docker-cqlsh cqlsh cassandra 9042 --cqlversion='3.4.4'
```

Debido a que el entorno Ubuntu utilizado no permite importar ficheros directamente, se creó manualmente una estructura en Cassandra y se insertó una muestra de registros del CSV. A continuación, se creó el keyspace:

```
CREATE KEYSPACE churn  
  
WITH replication = {'class':'SimpleStrategy',replication_factor:1};  
  
USE churn;
```

5. Creación de la tabla con los campos del dataset.

```
CREATE TABLE customers (  
    customer_id int PRIMARY KEY,  
    age int,  
    gender text,  
    tenure int,  
    usage_frequency int,  
    support_calls int,  
    payment_delay int,  
    subscription_type text,  
    contract_length text,  
    total_spend decimal,  
    last_interaction int,  
    churn_yesno text
```

);

```
cqlsh> USE churn;
cqlsh:churn> CREATE TABLE customers (
...     customer_id int PRIMARY KEY,
...     age int,
...     gender text,
...     tenure int,
...     usage_frequency int,
...     support_calls int,
...     payment_delay int,
...     subscription_type text,
...     contract_length text,
...     total_spend decimal,
...     last_interaction int,
...     churn_yn text
... );
```

6. Inserción manual de nueve registros del CSV, debido a que el entorno no permite subir ficheros directamente.

```
INSERT INTO customers VALUES (
401716,40,'Male',20,23,4,6,
'Standard','Quarterly',897.30,29,'No'
);
```

```
INSERT INTO customers VALUES (
99832,36,'Female',52,23,1,0,
'Standard','Quarterly',697.00,29,'Yes'
);
```

```
INSERT INTO customers VALUES (
168747,65,'Male',55,3,10,14,
'Basic','Quarterly',130.00,6,'Yes'
);
```

```
INSERT INTO customers VALUES (
126369,36,'Male',49,1,1,23,
'Standard','Annual',136.00,18,'Yes'
```

);

```
INSERT INTO customers VALUES (  
153212,55,'Male',8,10,8,24,  
'Standard','Monthly',709.00,13,'Yes'  
);
```

```
INSERT INTO customers VALUES (  
198312,22,'Male',3,14,4,28,  
'Premium','Annual',641.00,13,'Yes'  
);
```

```
INSERT INTO customers VALUES (  
424711,39,'Male',4,1,25,3,15,  
'Basic','Quarterly',683.52,10,'No'  
);
```

```
INSERT INTO customers VALUES (  
121600,36,'Male',38,18,7,25,  
'Basic','Monthly',441.00,2,'Yes'  
);
```

```
INSERT INTO customers VALUES (  
394723,44,'Female',54,23,0,11,  
'Standard','Annual',909.02,15,'No'
```

);

```
cqlsh:churn>
cqlsh:churn> INSERT INTO customers (
...     customer_id,
...     age,
...     gender,
...     tenure,
...     usage_frequency,
...     support_calls,
...     payment_delay,
...     subscription_type,
...     contract_length,
...     total_spend,
...     last_interaction,
...     churn_yesno
... )
... VALUES (
...     401716,
...     40,
...     'Male',
...     20,
...     23,
...     4,
...     6,
...     'Standard',
...     'Quarterly',
...     897.30,
...     29,
...     'No'
... );
cqlsh:churn> INSERT INTO customers (
...     customer_id, age, gender, tenure, usage_frequency, support_calls,
...     payment_delay, subscription_type, contract_length, total_spend,
...     last_interaction, churn_yesno
... )
... VALUES (
...     99832,36,'Female',52,23,1,0,'Standard','Quarterly',697.00,29,'Yes'
... );
cqlsh:churn> INSERT INTO customers (
...     customer_id, age, gender, tenure, usage_frequency, support_calls,
...     payment_delay, subscription_type, contract_length, total_spend,
...     last_interaction, churn_yesno
... )
... VALUES (
...     168747,65,'Male',55,3,10,14,'Basic','Quarterly',130.00,6,'Yes'
... );
```

Comprobación del número total de registros insertados:

SELECT COUNT() FROM customers;*

```
...     359957,28,'Male',34,15,1,1,'Basic','Quarterly',619.8,12,'NO
...     ... );
cqlsh:churn> SELECT COUNT(*) FROM customers;

count
-----
9

(1 rows)

Warnings :
Aggregation query used without partition key
cqlsh:churn> █
```

7. **Primera consulta:** clientes con tarifa Basic. El objetivo de esta consulta es recuperar y contar los registros cuyo campo Subscription Type es igual a Basic.

Contamos los clientes:

SELECT COUNT()*

FROM customers

WHERE subscription_type = 'Basic'

ALLOW FILTERING;

Resultado e interpretación: la consulta devuelve un total de 3 clientes con tarifa Basic dentro de la muestra insertada. Este resultado debe interpretarse únicamente como una comprobación funcional de la consulta, ya que la muestra contiene solo 9 registros y no representa el dataset completo.

```
Warnings :
Aggregation query used without partition key
cqlsh:churn> SELECT * FROM customers
... WHERE subscription_type='Basic'
... ALLOW FILTERING;
```

customer_id	age	churn_yesno	contract_length	gender	last_interaction	payment_delay	subscription_type	support_calls	tenure	total_spend	usage_frequency
121600	36	Yes	Monthly	Male	2	25	Basic	7	38	441.00	18
424711	39	No	Quarterly	Male	10	15	Basic	3	41	683.52	25
168747	65	Yes	Quarterly	Male	6	14	Basic	10	55	130.00	3

(3 rows)

8. **Segunda consulta:** clientes con tarifa Basic y contrato Annual. El objetivo es comprobar cuántos clientes cumplen simultáneamente ambas condiciones: Subscription Type = Basic y Contract Length = Annual.

Consulta para contar los clientes que cumplen ambas condiciones:

```
SELECT COUNT(*)
FROM customers
WHERE subscription_type = 'Basic'
AND contract_length = 'Annual'
ALLOW FILTERING;
```

Síntesis del anexo: se configuró Cassandra mediante Docker, se creó el keyspace churn y se definió una tabla adaptada al dataset de clientes. La inserción manual permitió validar el funcionamiento de las consultas, aunque los resultados deben entenderse como una prueba técnica y no como un análisis estadístico completo.

Anexo 9.2 Carga y tratamiento de datos con PySpark

En este anexo se describe el proceso de carga y preparación del dataset en Google Colab utilizando PySpark. El objetivo es importar el fichero CSV, crear una sesión

Spark, montar Google Drive y comprobar la estructura inicial de los datos antes de realizar los análisis posteriores.

En primer lugar, se cargará el fichero CSV en un DataFrame de PySpark, lo que permitirá realizar operaciones de agregación y análisis sobre los datos.

1. Instalación de PySpark en Google Colab.

```
!pip install pyspark
```

2. Importación de SparkSession para crear la sesión de trabajo en Spark.

```
from pyspark.sql import SparkSession
```

3. Importación del módulo necesario para acceder a Google Drive desde Google Colab.

```
from google.colab import drive
```

4. Montaje de Google Drive para acceder al fichero CSV almacenado en la cuenta del usuario.

```
drive.mount('/content/drive')
```

5. Creación de la sesión Spark, que permite ejecutar operaciones distribuidas sobre el DataFrame.

```
spark= SparkSession.builder.appName('Proyecto_final').getOrCreate()
```

+ Código

+ Texto

```
#Instalo el modulo Pyspark en google colab
!pip install pyspark

#Importo la sesion Spark
from pyspark.sql import SparkSession

#Importo el mdulo para acceder a google Drive
from google.colab import drive

#Monto Google drive
drive.mount('/content/drive')

#Creo la sesion Spark

spark= SparkSession.builder.appName("Proyecto_final").getOrCreate()
```

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.3)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (fr
Mounted at /content/drive

6. Importación del DataFrame desde el CSV almacenado en Drive. El archivo se carga indicando la coma como separador, y posteriormente se utilizan `show(5)` y `printSchema()` para revisar las primeras filas y comprobar los tipos de datos.

Con `df_customer.show(5)` se visualizan las cinco primeras filas del DataFrame.

Con `printSchema()` se muestran los tipos de datos de cada columna.

Anexo 9.3 Distribución de clientes, churn y duración de contratos

Este anexo analiza la distribución de clientes por tipo de tarifa y por duración del contrato, así como la tasa de churn asociada a cada categoría. La estructura seguida es: agrupación de datos, cálculo de porcentajes, generación de gráficos e interpretación de resultados.

1. Agrupación del número de clientes por tipo de tarifa.

```
df.groupBy('Subscription Type').count().show()
```

2. Cálculo de la tasa de churn por tipo de tarifa. Para ello, la variable "*Churn_YesNo*" se transforma en una variable numérica: Yes se convierte en 1 y No en 0. A continuación, se calcula la media de estos valores para obtener la proporción de clientes que abandonan la compañía en cada tarifa.

- Yes → 1
- No → 0

y calcula la media.

Importo las funciones necesarias

from pyspark.sql.functions import col, when, avg

Calculo el porcentaje de churn por tipo de tarifa.

Para ello convierto:

Yes -> 1

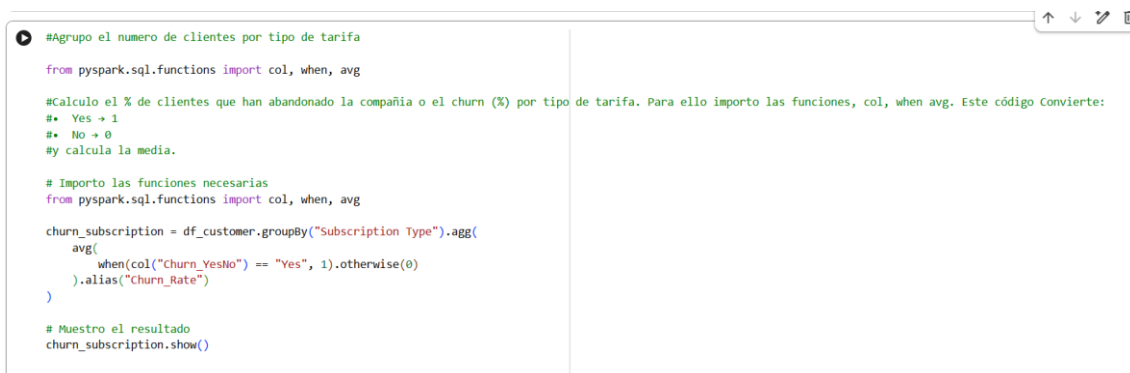
No -> 0

y posteriormente calculo la media.

```
churn_subscription = df_customer.groupBy('Subscription Type').agg(  
    avg(  
        when(col('Churn_YesNo') == 'Yes', 1).otherwise(0)  
    ).alias('Churn_Rate')  
)
```

Muestro el resultado

churn_subscription.show()



```
#Agrupo el numero de clientes por tipo de tarifa  
from pyspark.sql.functions import col, when, avg  
  
#Calculo el % de clientes que han abandonado la compañía o el churn (%) por tipo de tarifa. Para ello importo las funciones, col, when avg. Este código Convierte:  
#• Yes → 1  
#• No → 0  
#y calcula la media.  
  
# Importo las funciones necesarias  
from pyspark.sql.functions import col, when, avg  
  
churn_subscription = df_customer.groupBy("Subscription Type").agg(  
    avg(  
        when(col("Churn_YesNo") == "Yes", 1).otherwise(0)  
    ).alias("Churn_Rate")  
)  
  
# Muestro el resultado  
churn_subscription.show()
```

```

... +-----+-----+
|Subscription Type|    Churn_Rate|
+-----+-----+
|          Premium|0.5528479154433353|
|          Basic|0.5827404887101763|
|          Standard|0.5667955965486462|
+-----+-----+

```

Resultado e interpretación: la tarifa Premium presenta una tasa de churn del 55,28 %, la tarifa Basic del 58,27 % y la tarifa Standard del 56,67 %. Las diferencias entre tarifas son moderadas, por lo que el tipo de suscripción por sí solo no parece explicar de forma determinante el abandono de clientes.

3. Obtención del número total de clientes por tipo de tarifa.

```
from pyspark.sql.functions import count
```

```

clientes_tarifa = df_customer.groupBy('Subscription Type').agg(
    count('*').alias('Num_Clientes')
)

```

```
clientes_tarifa.show()
```

```

[5]
✓ 0 s #Calculo el numero de lcientes por tarifa

from pyspark.sql.functions import count

clientes_tarifa = df_customer.groupBy("Subscription Type").agg(
    count("*").alias("Num_Clientes")
)

clientes_tarifa.show()

```

```

... +-----+-----+
|Subscription Type|Num_Clientes|
+-----+-----+
|          Premium|          3406|
|          Basic|          3233|
|          Standard|          3361|
+-----+-----+

```

Resultado e interpretación: se observa que hay 3.596 clientes con tarifa Premium, 3.233 con tarifa Basic y 3.361 con tarifa Standard. La distribución es relativamente equilibrada, lo que permite comparar las tasas de churn entre tarifas sin una descompensación extrema en el tamaño de los grupos.

4. Generación del gráfico de clientes por tipo de tarifa. Antes de representarlo, el DataFrame de PySpark se convierte a Pandas, ya que Matplotlib trabaja directamente con estructuras compatibles con Pandas.

```
import matplotlib.pyplot as plt
```

```
subscription_pd = clientes_tarifa.toPandas()
```

```
plt.bar(
    subscription_pd['Subscription Type'],
    subscription_pd['Num_Clientes']
)
```

```
plt.xlabel('Tipo de tarifa')
```

```
plt.ylabel('Número de clientes')
```

```
plt.title('Clientes por tipo de tarifa')
```

```
plt.show()
```

```
[9]
✓ 0s #Procedo a generar el primer grafico de numero de clientes por tarifa importando la libreria matplotlib
#Antes de hacer el gráfico tengo que convertir el DataFrame de PySpark a Pandas.

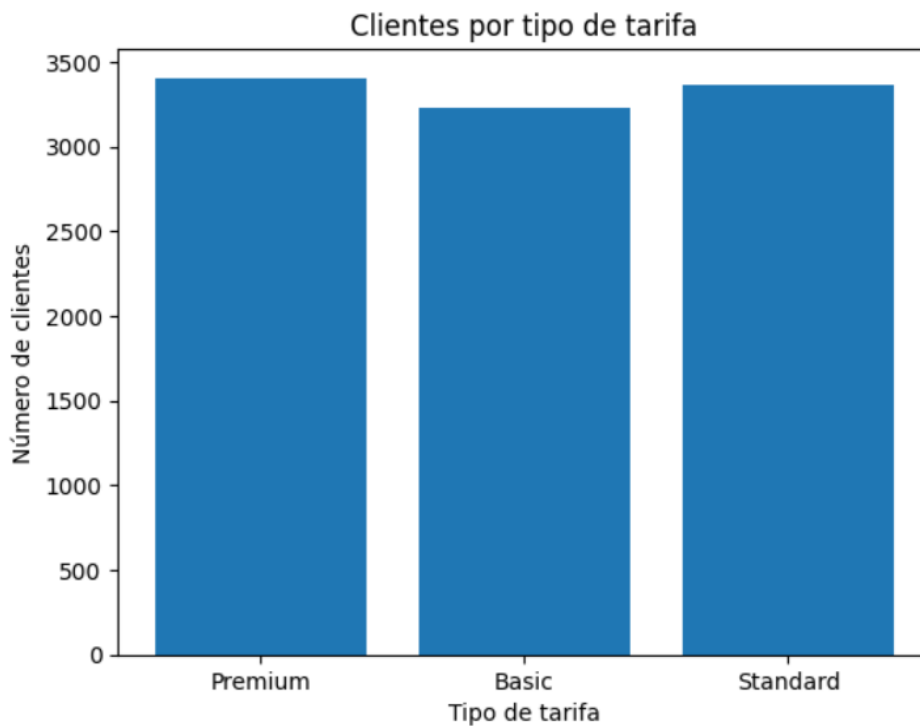
import matplotlib.pyplot as plt

subscription_pd = clientes_tarifa.toPandas()

plt.bar(
    subscription_pd["Subscription Type"],
    subscription_pd["Num_Clientes"]
)

plt.xlabel("Tipo de tarifa")
plt.ylabel("Número de clientes")
plt.title("Clientes por tipo de tarifa")

plt.show()
```



5. Generación del gráfico de churn por tipo de tarifa.

```
import matplotlib.pyplot as plt
```

```
subscription_pd = churn_subscription.toPandas()
```

```
plt.bar(
```

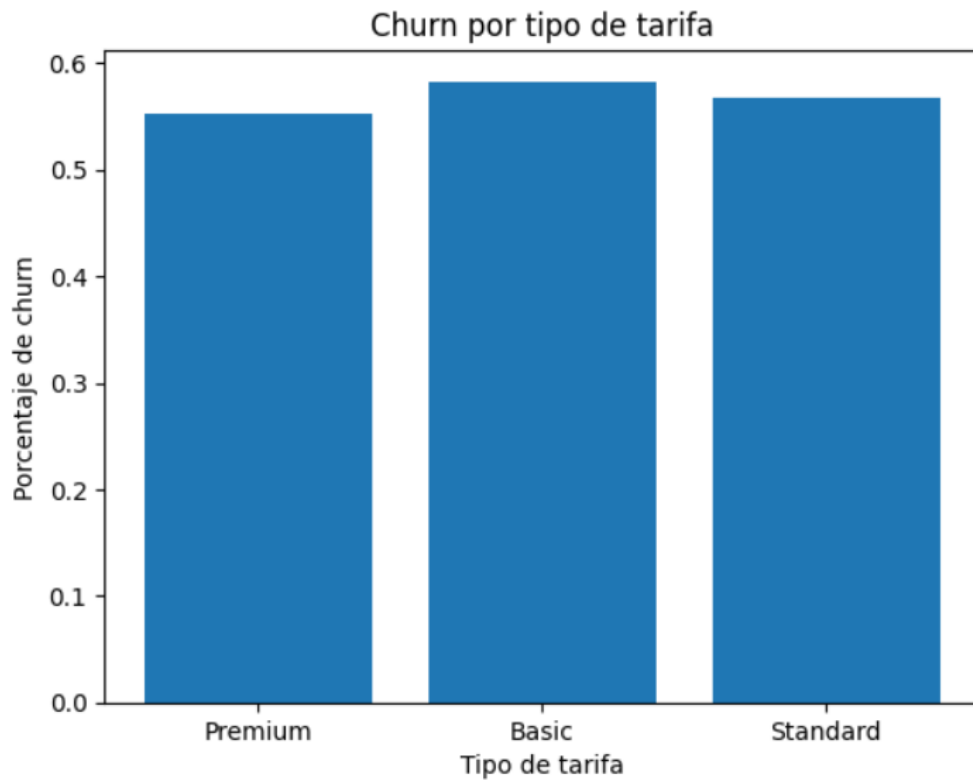
```
subscription_pd['Subscription Type'],  
subscription_pd['Churn_Rate']  
)
```

```
plt.xlabel('Tipo de tarifa')  
plt.ylabel('Porcentaje de churn')  
plt.title('Churn por tipo de tarifa')
```

```
plt.show()
```

▶ #A continuación genero el gráfico del churn por tipo de tarifa:

```
import matplotlib.pyplot as plt  
  
subscription_pd = churn_subscription.toPandas()  
  
plt.bar(  
    subscription_pd["Subscription Type"],  
    subscription_pd["Churn_Rate"]  
)  
  
plt.xlabel("Tipo de tarifa")  
plt.ylabel("Porcentaje de churn")  
plt.title("Churn por tipo de tarifa")  
  
plt.show()
```



6. Análisis por duración del contrato. Se repite el mismo procedimiento para la variable Contract Length. En primer lugar, se agrupa el número de clientes según la duración del contrato —Monthly, Quarterly y Annual— con el fin de conocer la composición de la base de clientes antes de analizar su relación con el abandono.

```
from pyspark.sql.functions import count
```

```
clientes_contrato = df_customer.groupBy('Contract Length').agg(  
    count('*').alias('Num_Clientes')  
)
```

```
clientes_contrato.show()
```

```
[7]
✓ 0 s
# Agrupo el número de clientes por duración del contrato

from pyspark.sql.functions import count

clientes_contrato = df_customer.groupBy("Contract Length").agg(
    count("*").alias("Num_Clientes")
)

clientes_contrato.show()
```

```
+-----+-----+
|Contract Length|Num_Clientes|
+-----+-----+
|      Annual   |         4046|
|    Quarterly  |         3970|
|     Monthly   |         1984|
+-----+-----+
```

Resultado e interpretación: la modalidad Annual es la más frecuente, con 4.046 clientes, seguida por los contratos Quarterly, con 3.970 clientes. Por último, los contratos Monthly representan el grupo menos numeroso, con 1.984 clientes.

7. Cálculo del porcentaje de abandono por tipo de contrato. Una vez conocido el número de clientes por modalidad de contrato, se calcula el porcentaje de abandono para cada una de ellas. Para ello, se convierte el valor Yes de la variable Churn_YesNo en 1 y el valor No en 0. Posteriormente, se calcula la media de estos valores mediante la función avg().

```
from pyspark.sql.functions import col, when, avg
```

```
churn_contract = df_customer.groupBy('Contract Length').agg(
    avg(
        when(col('Churn_YesNo') == 'Yes', 1).otherwise(0)
    ).alias('Churn_Rate')
)
```

```
churn_contract.show()
```

```
[8]
✓ 0 s      #Calculo el porcentaje de churn por duración del contrato

from pyspark.sql.functions import col, when, avg

churn_contract = df_customer.groupBy("Contract Length").agg(
    avg(
        when(col("Churn_YesNo") == "Yes", 1).otherwise(0)
    ).alias("Churn_Rate")
)

churn_contract.show()
```

Contract Length	Churn_Rate
Annual	0.4505684626791893
Quarterly	0.46977329974811083
Monthly	1.0

Resultado e interpretación: los contratos Annual presentan una tasa de churn del 45,06 %, mientras que los contratos Quarterly alcanzan el 46,98 %. En cambio, los contratos Monthly registran una tasa del 100 %, lo que indica que todos los clientes de esta modalidad abandonaron el servicio en el conjunto de datos analizado. Este resultado es especialmente relevante y sugiere que la contratación mensual puede estar asociada a una menor fidelización.

8. Visualización de la distribución de clientes por duración del contrato. Para generar el gráfico, se convierte el DataFrame de PySpark a Pandas, ya que la librería Matplotlib trabaja con este formato. Posteriormente, se genera un gráfico de barras que permite comparar el número de clientes en cada tipo de contrato.

```
import matplotlib.pyplot as plt
```

```
contract_pd = clientes_contrato.toPandas()
```

```
plt.bar(
    contract_pd['Contract Length'],
    contract_pd['Num_Clientes']
)
```

```
plt.xlabel('Duración del contrato')
```

```
plt.ylabel('Número de clientes')
```



```
plt.title('Clientes por duración del contrato')
```

```
plt.show()
```

```
▶ #Gráfico del churn por duración del contrato

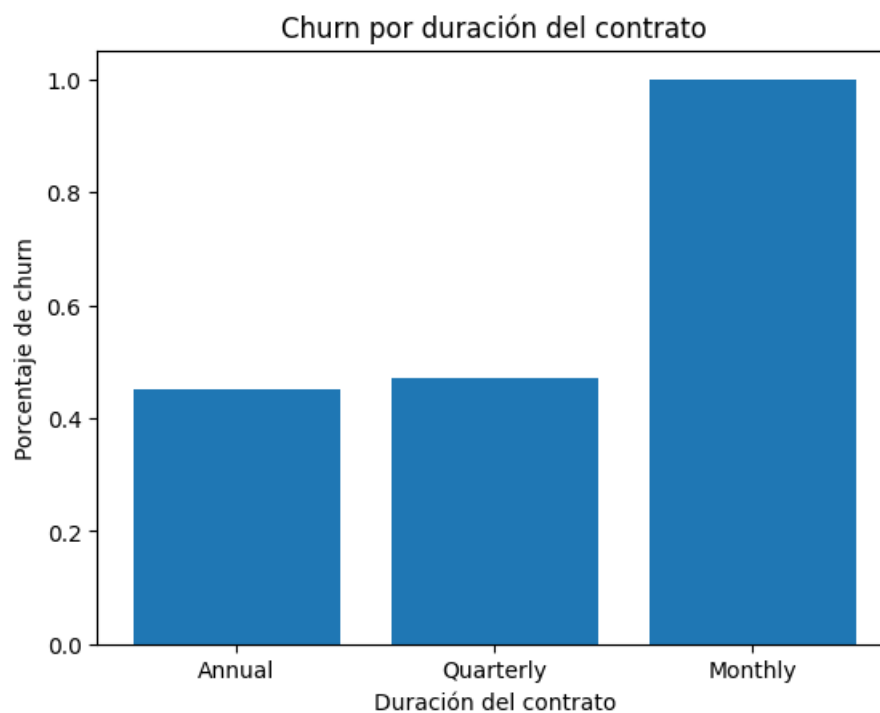
import matplotlib.pyplot as plt

contract_pd = churn_contract.toPandas()

plt.bar(
    contract_pd["Contract Length"],
    contract_pd["Churn_Rate"]
)

plt.xlabel("Duración del contrato")
plt.ylabel("Porcentaje de churn")
plt.title("Churn por duración del contrato")

plt.show()
```



9. Representación del porcentaje de abandono por tipo de contrato. Este gráfico permite comparar visualmente las diferencias en la tasa de churn entre contratos de distinta duración.

```
import matplotlib.pyplot as plt
```

```
contract_pd = churn_contract.toPandas()
```

```
plt.bar(  
    contract_pd['Contract Length'],  
    contract_pd['Churn_Rate']  
)
```

```
plt.xlabel('Duración del contrato')
```

```
plt.ylabel('Porcentaje de churn')
```

```
plt.title('Churn por duración del contrato')
```

```
plt.show()
```

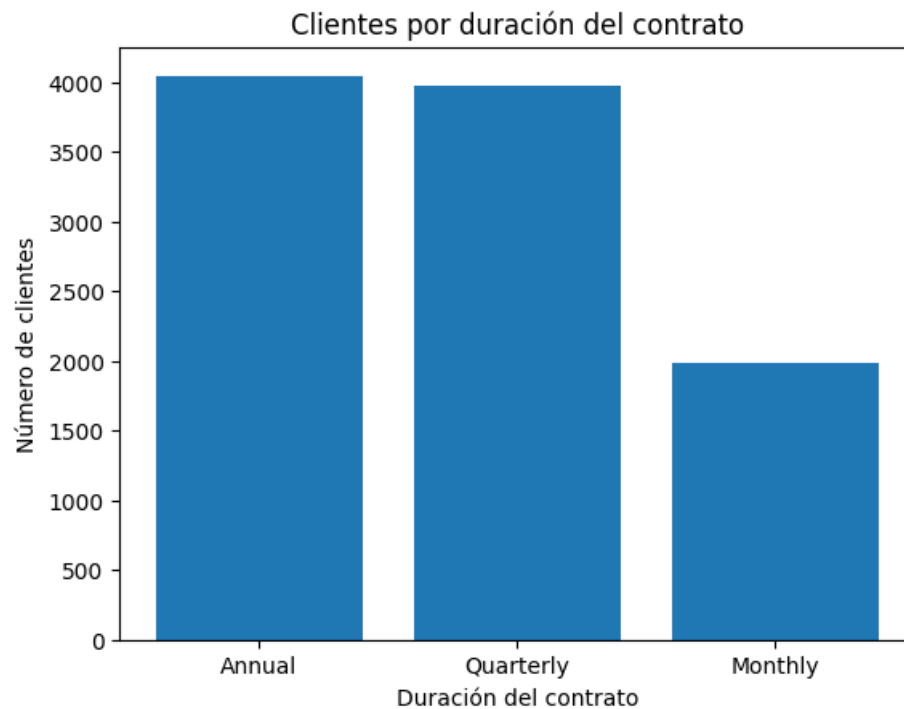
```
#Gráfico del número de clientes por duración del contrato  
import matplotlib.pyplot as plt
```

```
contract_pd = clientes_contrato.toPandas()
```

```
plt.bar(  
    contract_pd["Contract Length"],  
    contract_pd["Num_Clientes"]  
)
```

```
plt.xlabel("Duración del contrato")  
plt.ylabel("Número de clientes")  
plt.title("Clientes por duración del contrato")
```

```
plt.show()
```



Anexo 9.4 Análisis estadístico: media y desviación estándar

En este anexo se realiza un análisis descriptivo de las variables numéricas del dataset mediante el cálculo de la media y la desviación estándar. Se excluye CustomerID porque funciona como identificador único y no aporta información estadística relevante para interpretar el comportamiento de los clientes.

1. En primer lugar, se importan las funciones necesarias de PySpark:
mean(): permite calcular la media aritmética de una variable numérica.
stddev(): permite calcular la desviación estándar, que mide la dispersión de los datos respecto a la media:

```
from pyspark.sql.functions import mean, stddev
```

2. A continuación, se aplica el cálculo sobre las variables numéricas del DataFrame. Con el método select() se seleccionan las columnas del DataFrame y se aplican las funciones sobre ellas:

```
df_customer.select(  
  
    mean('Age').alias('Age_mean'),
```

```
stddev('Age').alias('Age_std'),

mean('Tenure').alias('Tenure_mean'),
stddev('Tenure').alias('Tenure_std'),

mean('Usage Frequency').alias('Usage_mean'),
stddev('Usage Frequency').alias('Usage_std'),

mean('Support Calls').alias('Support_mean'),
stddev('Support Calls').alias('Support_std'),

mean('Payment Delay').alias('PaymentDelay_mean'),
stddev('Payment Delay').alias('PaymentDelay_std'),

mean('Total Spend').alias('TotalSpend_mean'),
stddev('Total Spend').alias('TotalSpend_std'),

mean('Last Interaction').alias('LastInteraction_mean'),
stddev('Last Interaction').alias('LastInteraction_std')

).show()
```

Aplico estas funciones para las variables numéricas del dataset:

- **Age** → edad del cliente
- **Tenure** → antigüedad del cliente
- **Usage Frequency** → frecuencia de uso del servicio
- **Support Calls** → número de llamadas al soporte
- **Payment Delay** → retraso en pagos
- **Total Spend** → gasto total del cliente
- **Last Interaction** → última interacción con la empresa

```
[13]
✓ 1s #En este apartado se realiza un análisis descriptivo de las variables numéricas del conjunto de datos mediante PySpark.
#Para ello, se calcula la media y la desviación estándar de cada variable cuantitativa, excluyendo el campo CustomerID

from pyspark.sql.functions import mean, stddev

df_customer.select(

    mean("Age").alias("Age_mean"),
    stddev("Age").alias("Age_std"),

    mean("Tenure").alias("Tenure_mean"),
    stddev("Tenure").alias("Tenure_std"),

    mean("Usage Frequency").alias("Usage_mean"),
    stddev("Usage Frequency").alias("Usage_std"),

    mean("Support Calls").alias("Support_mean"),
    stddev("Support Calls").alias("Support_std"),

    mean("Payment Delay").alias("PaymentDelay_mean"),
    stddev("Payment Delay").alias("PaymentDelay_std"),

    mean("Total Spend").alias("TotalSpend_mean"),
    stddev("Total Spend").alias("TotalSpend_std"),

    mean("Last Interaction").alias("LastInteraction_mean"),
    stddev("Last Interaction").alias("LastInteraction_std")

).show()
```

Age_mean	Age_std	Tenure_mean	Tenure_std	Usage_mean	Usage_std	Support_mean	Support_std	PaymentDelay_mean	PaymentDelay_std
39.2613	12.492486883437296	30.9536	17.13792338300609	15.7726	8.631357904404563	3.5932	3.0796529327537288	13.0305	8.261863929499139

Interpretación de la captura: la tabla superior resume el valor medio y la dispersión de cada variable numérica. La media permite identificar el comportamiento promedio de los clientes, mientras que la desviación estándar ayuda a detectar qué variables presentan mayor variabilidad dentro del conjunto de datos.

Anexo 9.5 Análisis de correlación

En este anexo se desarrolla el análisis de correlación entre las variables numéricas del dataset utilizando PySpark. El objetivo es identificar si existen relaciones lineales relevantes entre las variables que puedan ayudar a interpretar el comportamiento de los clientes.

Para calcular la matriz de correlación es necesario transformar las variables numéricas en un único vector de características, ya que este es el formato requerido por las librerías de Machine Learning de Spark MLlib.

1. El primer paso será importar las librerías correspondientes.
 "VectorAssembler": une varias columnas en una única columna de tipo vector.
 "Correlation": calcula la matriz de correlación.

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.stat import Correlation
```

2. Se seleccionan las variables numéricas que se utilizarán para calcular la matriz de correlación.

```
columns = [  
    'Age',  
    'Tenure',  
    'Usage Frequency',  
    'Support Calls',  
    'Payment Delay',  
    'Total Spend',  
    'Last Interaction'
```

3. Se crea el vector de forma que cada variable quede integrada en una única columna. Spark necesita trabajar con un único vector. Después de esta transformación queda así:

```
[36,10,14,...]
```

En primer lugar, se crea un objeto `VectorAssembler`, encargado de combinar todas las variables numéricas del conjunto de datos en una única columna denominada `features`. Mediante el parámetro `inputCols` se especifican las columnas que deben incluirse en el vector, mientras que `outputCol` define el nombre de la nueva columna generada.

```
assembler = VectorAssembler(  
    inputCols=columns,  
    outputCol='features'  
)
```

```
df_vector = assembler.transform(df_customer).select('features')
```

4. Calculo la matriz de correlación.
"features": columna creada anteriormente.
"Pearson": coeficiente de Pearson.

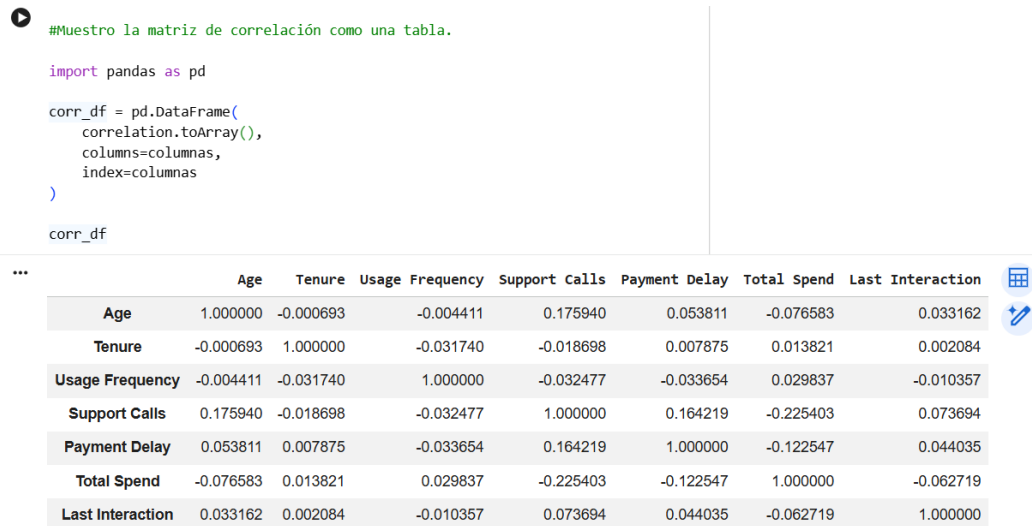
Finalmente, mediante [0][0] se accede a la única fila y columna del resultado, obteniendo la matriz de correlación ya que dentro de esa única celda está almacenada la matriz completa.

```
correlation = Correlation.corr(  
    df_vector,  
    'features',  
    'pearson'  
).collect()[0][0]  
  
print(correlation)
```

5. Para hacer más legibles los datos, la matriz de correlación se muestra como una tabla. Para ello, se convierte la matriz obtenida con PySpark en un DataFrame de Pandas, lo que facilita su visualización e interpretación.

En primer lugar, mediante `correlation.toArray()` se transforma el objeto de Spark que contiene la matriz de correlación en una estructura compatible con Pandas. Después, `pd.DataFrame()` crea una tabla en la que las columnas y las filas corresponden a las variables numéricas analizadas.

```
import pandas as pd  
  
corr_df = pd.DataFrame(  
    correlation.toArray(),  
    columns=columnas,  
    index=columnas  
)  
  
corr_df
```



Matriz de correlación

	Age	Tenure	Usage Frequency	Support Calls	Payment Delay	Total Spend	Last Interaction
Age	1.000.000	-0.000693	-0.004411	0.175940	0.053811	-0.076583	0.033162
Tenure	-0.000693	1.000.000	-0.031740	-0.018698	0.007875	0.013821	0.002084
Usage Frequency	-0.004411	-0.031740	1.000.000	-0.032477	-0.033654	0.029837	-0.010357
Support Calls	0.175940	-0.018698	-0.032477	1.000.000	0.164219	-0.225403	0.073694
Payment Delay	0.053811	0.007875	-0.033654	0.164219	1.000.000	-0.122547	0.044035
Total Spend	-0.076583	0.013821	0.029837	-0.225403	-0.122547	1.000.000	-0.062719
Last Interaction	0.033162	0.002084	-0.010357	0.073694	0.044035	-0.062719	1.000.000

Anexo 9.6 Decision Tree Classifier: árbol de decisión

En este anexo se construye un modelo de aprendizaje supervisado mediante un árbol de decisión. El objetivo es predecir la variable Churn_YesNo, que indica si un cliente abandona o no la compañía, utilizando tanto variables numéricas como variables categóricas previamente transformadas.

1. El primer paso será importar las librerías necesarias:
 - StringIndexer: convierte texto en números.
 - OneHotEncoder: transforma variables categóricas en variables binarias.

- VectorAssembler: une todas las variables en un único vector.
- DecisionTreeClassifier: crea el árbol de decisión.
- Pipeline: ejecuta todos los pasos automáticamente.
- MulticlassClassificationEvaluator: evalúa la precisión del modelo.

```
from pyspark.ml.feature import StringIndexer
from pyspark.ml.feature import OneHotEncoder
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import DecisionTreeClassifier
from pyspark.ml import Pipeline
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

2. El algoritmo de Machine Learning únicamente puede trabajar con datos numéricos. Sin embargo, el conjunto de datos incluye variables categóricas en formato texto, como Gender, Subscription Type, Contract Length y Churn_ YesNo, por lo que deben transformarse antes de entrenar el modelo.

Estas variables no pueden utilizarse directamente para entrenar el modelo. Para solucionarlo se utiliza StringIndexer, que asigna un número diferente a cada categoría. Por ejemplo, la variable Gender podría quedar transformada de la siguiente manera:

Gender	Gender_Index
Male	0
Female	1

```
gender = StringIndexer(
    inputCol='Gender',
    outputCol='Gender_Index'
)
```

```
subscription = StringIndexer(
    inputCol='Subscription Type',
    outputCol='Subscription_Index'
)
```

```
contract = StringIndexer(
```

```
inputCol='Contract Length',
outputCol='Contract_Index'
```

```
)
```

```
[14]
✓ 0 s
```

```
#Convierto las variables categóricas en numericas
```

```
gender_indexer = StringIndexer(
    inputCol="Gender",
    outputCol="Gender_Index"
)
```

```
subscription_indexer = StringIndexer(
    inputCol="Subscription Type",
    outputCol="Subscription_Index"
)
```

```
contract_indexer = StringIndexer(
    inputCol="Contract Length",
    outputCol="Contract_Index"
)
```

```
label_indexer = StringIndexer(
    inputCol="Churn_YesNo",
    outputCol="label"
)
```

En el caso de *Churn_YesNo*, la nueva columna recibe el nombre *label*, ya que Spark utiliza ese nombre por defecto para identificar la variable que se desea predecir.

```
churn = StringIndexer(
    inputCol='Churn_YesNo',
    outputCol='label'
)
```

3. Aplico "OneHotEncoder": Aunque las variables categóricas ya se han convertido en números mediante "StringIndexer", esos números únicamente sirven para identificar cada categoría.

Ejemplo:

Tarifa	Número
Basic	0
Standard	1

Tarifa	Número
Premium	2

Si se utilizaran directamente estos valores, el modelo podría interpretar incorrectamente que Premium (2) es mayor que Standard (1), cuando realmente no existe ninguna relación de orden entre ellas.

Para evitar este problema se utiliza "OneHotEncoder", que transforma cada categoría en una representación binaria (vector de 0 y 1), eliminando cualquier interpretación numérica entre las distintas categorías.

```
encoder = OneHotEncoder(
```

```
    inputCols=[
        'Gender_Index',
        'Subscription_Index',
        'Contract_Index'
    ],
```

```
    outputCols=[
        'Gender_Vector',
        'Subscription_Vector',
        'Contract_Vector'
    ]
)
```

```
[15] #Utilizo "OneHotEncoder", que transforma cada categoría en una representación binaria (vector de 0
#eliminando cualquier interpretación numérica entre las distintas categorías.

encoder = OneHotEncoder(

    inputCols=[
        "Gender_Index",
        "Subscription_Index",
        "Contract_Index"
    ],

    outputCols=[
        "Gender_Vector",
        "Subscription_Vector",
        "Contract_Vector"
    ]

)
```

4. Uno las variables, ya que los algoritmos de Machine Learning de Spark necesitan que toda la información de cada cliente esté agrupada en una única columna denominada que llamaré "features". Para ello utilizaré

"VectorAssembler", que toma todas las variables numéricas y las variables categóricas previamente codificadas y las reúne en un único vector.

```
assembler = VectorAssembler(
```

```
    inputCols=[
```

```
        'Age',
```

```
        'Tenure',
```

```
        'Usage Frequency',
```

```
        'Support Calls',
```

```
        'Payment Delay',
```

```
        'Total Spend',
```

```
        'Last Interaction',
```

```
        'Gender_Vector',
```

```
        'Subscription_Vector',
```

```
        'Contract_Vector'
```

```
    ],
```

```
    outputCol='features'
```

```
)
```

```
[16] ✓ 0 s
#Creo el vector para reunir todas las variables
assembler = VectorAssembler(

    inputCols=[

        "Age",
        "Tenure",
        "Usage Frequency",
        "Support Calls",
        "Payment Delay",
        "Total Spend",
        "Last Interaction",

        "Gender_Vector",
        "Subscription_Vector",
        "Contract_Vector"

    ],

    outputCol="features"

)
```

5. Creo el árbol de decisión. En este paso se crea el algoritmo de clasificación.

```
tree = DecisionTreeClassifier(
```

```
featuresCol='features',
```

```
labelCol='label'
```

```
)
```

```
[17] ✓ 0 s # Creo el árbol de decisión.
dt = DecisionTreeClassifier(

    labelCol="label",
    featuresCol="features"
)
```

6. Se crea el Pipeline: El Pipeline permite automatizar todas las transformaciones necesarias antes de entrenar el modelo. Cuando Spark ejecute el Pipeline realizará automáticamente los siguientes pasos:
- Convertirá las variables categóricas mediante **StringIndexer**.
 - Aplicará **OneHotEncoder**.
 - Construirá la columna **features** mediante **VectorAssembler**.
 - Entrenará el árbol de decisión.

```
pipeline = Pipeline(
```

```
    stages=[
```

```
        gender,
```

```
        subscription,
```

```
        contract,
```

```
        churn,
```

```
        encoder,
```

```
        assembler,
```

```
        tree
```

```
]
)
```

```
[18]
✓ 0 s

# Creo el Pipeline

pipeline = Pipeline(

    stages=[

        gender_indexer,
        subscription_indexer,
        contract_indexer,
        label_indexer,

        encoder,

        assembler,

        dt

    ]

)
```

7. Antes de entrenar el modelo es necesario dividir el conjunto de datos en dos partes.

- El **80 %** de los registros se utilizará para entrenar el modelo (train).
- El **20 %** restante se reservará para comprobar posteriormente la calidad de las predicciones (test).

El parámetro "seed" fija una semilla aleatoria para que la división sea siempre la misma al volver a ejecutar el código, facilitando la reproducibilidad de los resultados.

```
train, test = df_customer.randomSplit([0.8,0.2], seed=123)
```

```
[19]
✓ 0 s

#Divido el dataset para la prueba y el entrenamiento

train, test = df_customer.randomSplit(

    [0.8,0.2],

    seed=123

)
```

8. Para ver qué configuración del árbol es mejor, se utiliza GridSearch, que permite probar diferentes configuraciones del modelo. En este caso, se prueban árboles con profundidades 3, 5, 7 y 10. Antes de entrenar el modelo, se importa la herramienta ParamGridBuilder.

```
from pyspark.ml.tuning import ParamGridBuilder
```

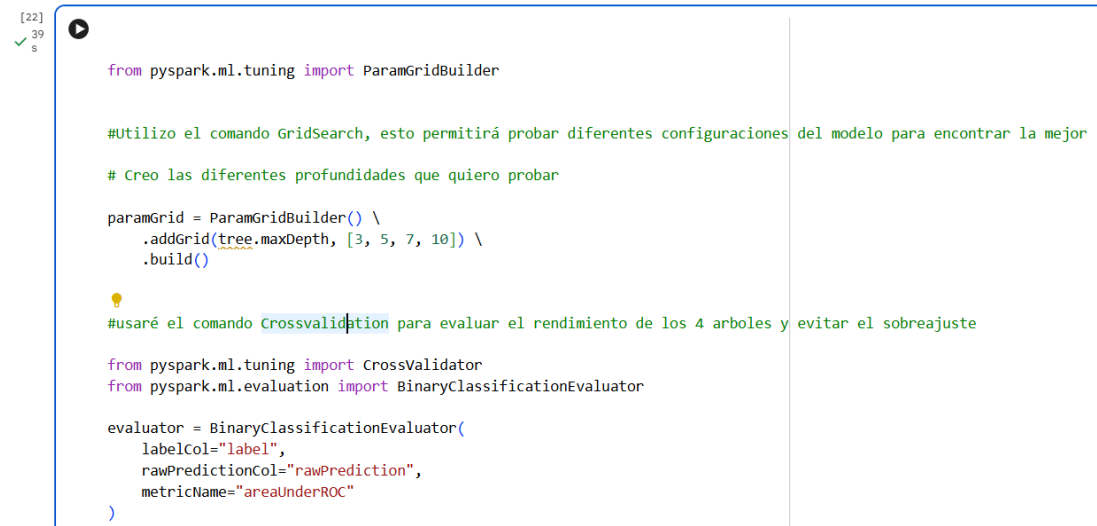
```
paramGrid = ParamGridBuilder() \
    .addGrid(tree.maxDepth, [3, 5, 7, 10]) \
    .build()
```

9. A continuación, se utiliza CrossValidator para evaluar el rendimiento de los cuatro árboles y evitar el sobreajuste, que ocurre cuando el modelo memoriza los datos de entrenamiento en lugar de aprender patrones generales. Para ello, se importa CrossValidator y se evalúa el modelo con la métrica AUC mediante BinaryClassificationEvaluator.

```
from pyspark.ml.tuning import CrossValidator
```

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator
```

```
evaluator = BinaryClassificationEvaluator(
    labelCol='label',
    rawPredictionCol='rawPrediction',
    metricName='areaUnderROC'
)
```



```
[22] ✓ 39 s
from pyspark.ml.tuning import ParamGridBuilder

#Utilizo el comando GridSearch, esto permitirá probar diferentes configuraciones del modelo para encontrar la mejor
# Creo las diferentes profundidades que quiero probar

paramGrid = ParamGridBuilder() \
    .addGrid(tree.maxDepth, [3, 5, 7, 10]) \
    .build()

#usaré el comando Crossvalidation para evaluar el rendimiento de los 4 arboles y evitar el sobreajuste

from pyspark.ml.tuning import CrossValidator
from pyspark.ml.evaluation import BinaryClassificationEvaluator

evaluator = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)
```

Se crea el Pipeline, se prueban los diferentes árboles y con CrossValidator se elige el mejor.

```

crossval = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=paramGrid,
    evaluator=evaluator,
    numFolds=5,
    seed=123
)

```

10. Se entrena el modelo.

```

modelo_cv = crossval.fit(train)

```

```

#Creo el CrossValidation

crossval = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=paramGrid,
    evaluator=evaluator,
    numFolds=5,
    seed=123
)

#Entreno el modelo

modelo_cv = crossval.fit(train)

```

11. Para averiguar qué árbol se ha elegido:

```

best_tree = modelo_cv.bestModel.stages[-1]

print('Mejor maxDepth:', best_tree.getMaxDepth())

```

12. **Generación de predicciones.** Una vez entrenado el modelo, se aplica sobre el conjunto de prueba mediante el método "train" y "test".

```

pred_train = modelo_cv.transform(train)

pred_test = modelo_cv.transform(test)

```



```

1]  #Generación de predicciones. Una vez entrenado el modelo, se aplica sobre el conjunto de prueba mediante el método "train" y "test"
0 s

best_tree = modelo_cv.bestModel.stages[-1]
print("Mejor maxDepth:", best_tree.getMaxDepth())

Mejor maxDepth: 5

```

13. Cálculo de la precisión del modelo. La métrica accuracy representa el porcentaje de clientes correctamente clasificados por el árbol de decisión. Un valor próximo a 1 indica un elevado nivel de acierto, mientras que un valor cercano a 0 refleja un rendimiento deficiente.

```

from pyspark.ml.evaluation import MulticlassClassificationEvaluator

```

```

accuracy_evaluator = MulticlassClassificationEvaluator(

```

```

    labelCol='label',

```

```

    predictionCol='prediction',

```

```

    metricName='accuracy'

```

```

)

```

```

accuracy_test = accuracy_evaluator.evaluate(pred_test)

```

```

print('Accuracy Test:', accuracy_test)

```

```

[26]  #Realizo las predicciones
✓ 0 s

pred_train = modelo_cv.transform(train)
pred_test = modelo_cv.transform(test)

[27]  #Calculo el accuracy
✓ 0 s

from pyspark.ml.evaluation import MulticlassClassificationEvaluator

accuracy_evaluator = MulticlassClassificationEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="accuracy"
)

accuracy_test = accuracy_evaluator.evaluate(pred_test)

print("Accuracy Test:", accuracy_test)

... Accuracy Test: 0.9621062992125984

```

Resultado e interpretación: una accuracy de 0,9621 indica que el modelo clasificó correctamente aproximadamente el 96,21 % de los casos del conjunto de prueba.

14. A continuación, se evalúa la precisión del modelo con AUC:

```
auc_test = evaluator.evaluate(pred_test)
```

```
print('AUC Test:', auc_test)
```

```
[28]
✓ 0 s      #A continuacion evaluo la precisión del modelo con AUC:

          auc_test = evaluator.evaluate(pred_test)

          print("AUC Test:", auc_test)

▼          AUC Test: 0.9666226867464541
```

El modelo obtuvo un valor de AUC de 0,9666. Este resultado, cercano a 1, indica una capacidad muy elevada para distinguir entre los clientes que abandonan y los que permanecen en la compañía.

15. Se representa gráficamente la curva ROC, que muestra la capacidad de discriminación del modelo para distinguir entre clientes que abandonan y clientes que no abandonan. Con la curva ROC se puede ver si el modelo ordena correctamente a los clientes en función de su riesgo de churn.

```
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt
```

Obtengo los datos para test

```
# Cogemos realidad + probabilidades
roc_test = pred_test.select(
    'label',
    'probability'
).toPandas()
```

```
# ¿Qué ocurrió realmente?
```

```
y_test = roc_test['label']
```

```
# ¿Qué probabilidad de churn dio el modelo?
```

```
y_score_test = roc_test['probability'].apply(
```

```
    lambda x: float(x[1])
```

```
)
```

```
# Comparamos realidad y probabilidades
```

```
# para obtener los puntos de la curva ROC
```

```
fpr_test, tpr_test, _ = roc_curve(
```

```
    y_test,
```

```
    y_score_test
```

```
)
```

```
# Dibujamos la curva
```

```
plt.plot(
```

```
    fpr_test,
```

```
    tpr_test,
```

```
    label=f'Test AUC = {auc_test:.3f}'
```

```
)
```

[29]
✓ 0 s

```
#Calculo la curva ROC
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

roc_test = pred_test.select(
    "label",
    "probability"
).toPandas()

y_test = roc_test["label"]
y_score_test = roc_test["probability"].apply(lambda x: float(x[1]))

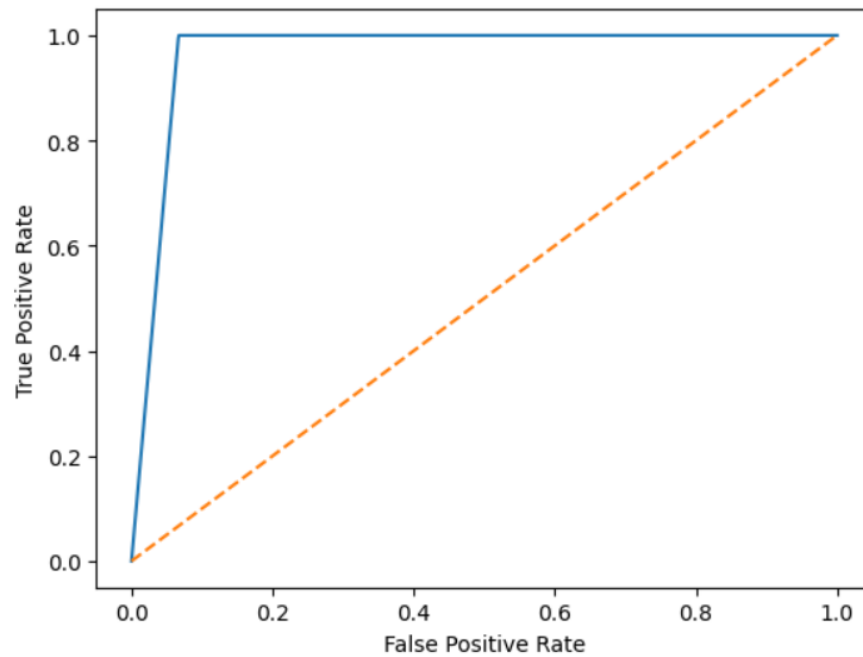
fpr_test, tpr_test, _ = roc_curve(
    y_test,
    y_score_test
)

#Dibujo la curva ROC

plt.plot(
    fpr_test,
    tpr_test
)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```
plt.plot(
    [0,1],
    [0,1],
    linestyle="--",
    label="Random"
)
```

... [matplotlib.lines.Line2D at 0x7a3d888651d0>]



Estos son los resultados del código en Google Colab: AUC Test: 0.9666226867464541 y Accuracy Test: 0.9621062992125984.

La gráfica muestra la relación entre la tasa de verdaderos positivos y la tasa de falsos positivos, con valores entre 0,0 y 1,0. En el eje X, la curva alcanza aproximadamente el valor 0,05, mientras que en el eje Y llega a 1. A partir de ese punto, la recta ya no crece más en el eje Y.